AARHUS UNIVERSITY

DEPARTMENT OF ENGINEERING

COMPUTER ENGINEERING

Term:	FALL 2021
Examination:	Introduction to Programming for Computer Engineers
Date:	19 January 2022
Duration:	3 hours, 9:00 – 12:00
Room:	room number(s)

1 cover plus paper for draft notes will be handed out.

REMEMBER to state name and study number on all pages of your submission.

Remember to show us an ID when returning the present exam form. We will then mark your name on our master list.

- This document is split into 5 sections focusing on different topics of the lecture. Each section contains up to 5 questions. You have 180 minutes to answer all of them. In addition, there is one challenging bonus question that you can work on if you get the time, since it will be possible to get the highest mark even without answering the bonus question, but it can compensate for mistakes you have done in the first 5 sections. Strategically, you should first scan it quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are multiple choice questions, questions that require you to write C code and drawings. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt what is expected, please ask!**
- The multiple choice questions may have any combination of correct answers. All choices may be correct, none or anything in between.
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.
- At the end of the exam paper (Section 6) you find one bonus question worth 10 points. You are ***not*** required to solve it and can attain the full score of 100 points ***without*** it. It is harder to solve but it can be used to compensate for mistakes in answers to other questions.

1 Types and Syntax

Question 1.1 **[3]**

What type do the following values belong to? Indicate your answer by ticking the boxes on the right.

	`float`	`float[2]`	`char`	`char[10]`
• `2.2`	☒	☐	☐	☐
• `'2'`	☐	☐	☒	☐
• `"2.2"`	☐	☐	☐	☒
• `{ -4.0, 3.6 }`	☐	☒	☐	☐

Question 1.2 **[3]**

If a program is compiled and the result of the compilation is exit code 0. Does it mean that (tick the correct box):

☐ The program could not compile

☒ The program is compiled, but errors can potentially occur when it is run

☐ The program is compiled and will run with no errors

Question 1.3 **[3]**

A function `volume` calculates and returns the volume of a pyramid as a float. The input is given as three floats: `l`, `w`, `h`, where `l` and `w` are the length and width of the base, and `h` is the height. Complete the function signature below.

```
float volume ( float l, float w, float h ) {
  return (1/3)*l*w*h;
}
```

SOLUTION REMARK: Order of input arguments does not matter, any order is correct.

Question 1.4 **[3]**

The function `increment` takes a pointer to an integer and multiplies the integer value which it points to with `3`. The function does not return any value. Complete the function signature and function body below, so that the main function prints `15` when executed.

```
void increment ( int* v ) {
    *v = *v * 3;
}

int main () {
  int v = 5;
  increment(&v);
  printf("%d", v);
  return 0;
}
```

Question 1.5 **[5]**

Fill in the type definition below for a struct defining a type called `person`. It must contain the fields `age` of type `int`, `name` of type `char` pointer, and `sex` of type `char`.

```
typedef struct {
    int age;
    char* name;
    char sex;
} person;
```

SOLUTION REMARK: Order of data elements does not matter, any order is correct.

2 Basic Programs

Question 2.1 **[3]**

Let `q`, `w`, `t` be integer variables declared by

```
int q;
int w;
int t;
```

Which assignment (a), (b), (c) or (d) inserted for the question mark in the program fragment below makes the following program fragment print out `5`:

```
  q = 1;
  w = 4;
  ?
  t = q + w;
  printf("%d", t);
```

Answer:

(a) `w = q * w;` ☒
(b) `q = q * w;` ☐
(c) `w = q + w;` ☐
(d) `q = w + q;` ☐

Tick the box on the right of the assignment to indicate by which assignment to replace the question mark in the program fragment. More than one expression may be correct.

Question 2.2 **[3]**

Consider the following program fragment

```
  int i;
  int b[] = {0,1,1,2,3,5,8,13};
  for(i = 0; i < 7; i = i + 1)
    b[i] = b[i] + b[i+1];
```

Fill in the table below with the values stored in array `b` after the program has been executed.

Answer: `b` =

1	2	3	5	8	13	21	13

Question 2.3 **[4]**

Consider this program fragment

```
int c = a*b;
int d = 1;
if(b == 1)
  c = 1;
if(a == 1 || b == 0)
  d = 1;
d = c * d;
printf("%d", d);
```

For the values of variables `a` and `b` as indicated below what is the resulting value of variable `d` (i.e. what is printed out)?

a	b	d				
1	1	-1 ☐	0 ☐	1 ☒	2 ☐	someting else ☐
1	0	-1 ☐	0 ☒	1 ☐	2 ☐	someting else ☐
0	1	-1 ☐	0 ☐	1 ☒	2 ☐	someting else ☐
0	0	-1 ☐	0 ☒	1 ☐	2 ☐	someting else ☐

Tick the box next to the correct value for `d` with respect to the initial values of `a` and `b` as stated in the first two columns of the table.

Question 2.4 **[4]**

When compiling the following function:

```
6   typedef struct node{
7       int value;
8       struct node* next;
9   }node;
10
11  int sum_squares(node* p) {
12    if (p==NULL)
13      p->value = 0;
14    else
15      return (p->value*p->value + sum_squares(p->next));
16  }
```

the compiler gave the following warning:

```
main.c:16:1: warning: control may reach end of non-void function [-Wreturn-type]
}
^
1 warning generated.
```

What is the warning and how will you change to program so that it can compile without warnings (include the line number(s) where you will make the change(s))?

The warning is: Hvis (p==NULL) er sand returnerer funktionen intet, den skal returnere en int

It is corrected by: ændre linie 13 til return 0

SOLUTION REMARK: Many different explanations are possible and accepted. Several changes could also be accepted

Question 2.5 **[6]**

Consider the program fragment below.

```
int x;
int y;
int z;

x = ?
y = ?
z = ?

switch(x) {
  case 0:
    x = y*z;
    z = z*2;
    break;
  case 1:
    y = x + z;
    break;
  default:
    {
      int z;
      z = x + y;
      x = y * z;
    }
    break;
}

printf("[%d, %d, %d]", x, y, z);
```

For each of the following assignments for variables `x`, `y`, `z`, write the three integer numbers that will be printed when the program is executed.

(1) `x=0`, `y=1`, `z=2`:

Answer: [2 , 1 , 4]

(2) `x=2`, `y=3`, `z=4`:

Answer: [15 , 3 , 4]

3 While Programs

Question 3.1 **[6]**

You are given the declarations of variables `w` and `i`, and the initialisation of them.

```
int i;
int w;
w = 2;
i = 2;
```

You are given the following while-loop:

```
while(w < 10) {
  w = w + 1;
  i = i + 2;
}
```

Fill in the blanks below so that the resulting for-loop computes the same values for `w` and `i` as the while-loop above. You are only allowed to use variable declarations, assignments and conditions that already occur in the while-loop.

```

int i;
_____

int w;
_____

i = 2;
_____


      w = 1;     w < 10;     w = w + 1
for (_____________________________________________) {

  i = i + 2;
  ____________________________________

}
```

SOLUTION REMARK: This is **not** the only valid solution, more than one solution is acceptable as long as (a) w< 10 is the only *for loop* condition (b) w, i are declared and initialised properly (c) both w and i are incremented each loop iteration (by 1 and 2.

Question 3.2 **[6]**

What do the following program fragments print out at the end of their execution where `i` and `x` are integer variables?

Program fragment 1:

```
x = 2;
for (i=0; i < 3; i = i + 1)
  x = i*2;
printf("%d", x);
```

Program fragment 2:

```
x = 3;
for (i=1; i > 0; i = i + 1)
  i = i - 2;
printf("%d", x);
```

Program fragment 3:

```
x = -2;
for (i=0; i < 8; i = i + 1)
  x = x + 1;
printf("%d", x);
```

Indicate your answer by ticking the column with the correct result in the table below. The column 'no result' signifies that no value is printed at all (indefinite loop).

	1	2	3	4	5	6	7	8	9	no result
Program fragment 1				X						
Program fragment 2			X							
Program fragment 3						X				

Question 3.3 **[6]**

Consider the following program fragment.

```
int x;
int f;
int fi;

fi = 1;
f  = 2;

scanf("%d", &x);

if(x <= 1)
  abort();

_______________________  /* Precondition goes here */

while(f < x) {

  _______________________  /* Loop Invariant goes here */

  fi = f;
  f  = fi + 1;
}

_______________________  /* Postcondition goes here */
```

Tick the following boxes such that the property specified for the column holds (as an assertion) at the program location specified by the row. Tick all true statements for each row (from 0 to 6 ticks).

	fi < f	f < fi	0 < x	x < 0	(f-1) < x <= f	x>=f
Precondition	☒	☐	☒	☐	☐	☒
Loop Invariant	☒	☐	☒	☐	☐	☒
Postcondition	☒	☐	☒	☐	☒	☐

Question 3.4 **[8]**

For each program below, let N be any positive integer. After executing each program, what is the value of v as a function of N?

Program 1:

```
int v = 0;
for(int i = 1; i <= N; i = i + 1) {
  v = v + 2;
}
```

Program 2:

```
int v = 0;
for(int j = 1; j <= N; j = j + 1) {
  for(int i = 1; i <= N; i = i + 1) {
    v = v + 1;
  }
}
```

Program 3:

```
int v = 0;
for(int i = 1; i <= N*N; i = i + 1) {
    v = v + 1;
}
```

Program 4:

```
int i;
int v = 0;
for(i = 1; i <= N; i = i + 1) {
  v = v + 1;
}
v=0;
for(i = 1; i <= N; i = i + 1) {
  v = v + 1;
}
```

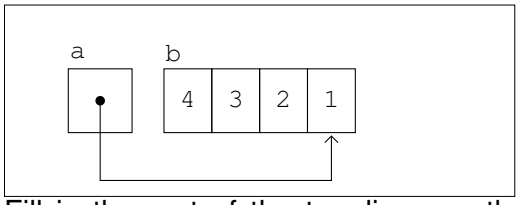
Tick the boxes below to indicate the value of v as a function of N (column) for each program (row). *Hint*: exactly one answer is correct per program.

	v=N	v=2*N	v=N^2	v=$\frac{N(N-1)}{2}$	2^N
Program 1	☐	☒	☐	☐	☐
Program 2	☐	☐	☒	☐	☐
Program 3	☐	☐	☒	☐	☐
Program 4	☒	☐	☐	☐	☐

4 Pointer Programs

Question 4.1 [3]

The following pictures (in the notation from the lecture) shows two variables (a and b) and their values.



Fill in the rest of the two lines so that the declaration and assignment results in the illustration:

```
int b__[4]= ______{4, 3, 2, 1}_________ ;

int*a____ = &b[3]_____________________;
```

Hint: Some part of the lines might be empty

Question 4.2 [4]

Consider the following program fragment:

```
  int x;
  int y;
  int z;
  int* w;
  int* q;
  x = 0;
  y = 1;
  z = 2;
  w = &y;
  q = &y;
 *w = y;
 *q = z;
 *w = x + y + z + *q;
 *q = x + y + z + *w;
 printf("x=%d, y=%d, z=%d",x,y,z);
```

What does the program print when it is executed?

Answer: x=0, y=14, z=2

Question 4.3 **[6]**

Consider the following program fragment:

```
1  typedef struct node {
2     int data;
3     struct node* follower;
4  } node;
5
6
7  node* new_node(int v, node* l)
8  {
9   node *n = (node*)malloc(sizeof(node));
10  n->data = v;
11  n->follower = l;
12  return n;
13 }
14
15 node* rt(node* n) {
16  if(n->follower == NULL) {
17    return NULL;
18  }
19  else {
20     return new_node(n->data, n);
21  }
22 }
23
24 void print(node* n){
25   if (n!=NULL) {
26     printf("%d, ",n->data);
27     print(n->follower);
28   } else
29     printf("\n");
30 }
31
32 int main () {
33   node* a = new_node(1,
34                      new_node(2, NULL));
35   a = rt(a);
36   print(a);
37   a = rt(a);
38   print(a);
39 }
```

What is the output of the program?

line 36: 1, 1, 2,

line 38: 1, 1, 1, 2,

5 Recursive Programs

Question 5.1 **[4]**

You are given the following program fragment

```
int recur(int n){
  assert(n>=0);
  int t = 2;
  if (n==0)
    return t;
  else
    return t+ recur(n-1);
}

int main(void) {
  printf("recur(3)=%d", recur(3));
}
```

What does it print out? Tick the corresponding box below.

- ☐ `recur(3)=0`
- ☐ `recur(3)=1`
- ☐ `recur(3)=2`
- ☐ `recur(3)=4`
- ☐ `recur(3)=6`
- ☒ `something other than the above`

Question 5.2 **[4]**

What is the missing assertion and base case of the following function for computing the sum, i.e. the sum of integers from `0` to `n` by calling, for instance, `k = sum(n);`?

```
int sum(int n) {
    ________________
    ________________
    else {
        return n + sum(n - 1);
    }
}
```

Tick the appropriate program fragment to complete the if-statement in function `sum`.

- ☐ `assert(n == 0);`
  `if(n == 0) return 0;`
- ☒ `assert(n >= 0);`
  `if(n == 0) return 0;`
- ☐ `assert(n == 1);`
  `if(n == 1) return 1;`
- ☐ `assert(n >= 1);`
  `if(n == 1) return 1;`

Question 5.3 **[8]**

You are given the following declaration of a tree datatype, where the empty tree corresponds to the pointer value `NULL`, the field `data` stores the value at a node of the tree, and `left`, `right`, are the left and right children.

```
typedef struct node
{
  int data;
  struct node* left;
  struct node* right;
} node;
```

Complete the missing code in the recursive `product` function so that it returns the product of all nodes in the tree `t`.

```
int product(node *t) {


  if( ____t==NULL____________________________ )

          1
    return ____________________________________ ;


  else

          t->data * product(t->left) * product(t->right)
    return ____________________________________ ;
}
```

You can assume that no overflow will occur.

Question 5.4 **[8]**

Write a *recursive* search function `search` by completing the blanks. The input is an array of integers, an integer (the size of the array) and an integer to be search for (called `target`). The function returns true if `target` is in the array, false otherwise.

```
#include <stdbool.h>

bool search( int a[] , int n , int target) {

  assert( n>=0 );

  if( n==0 )

    return false;

  else if ( n==1 )

    return a[0]==target;

  else

    return a[n-1]==target ||search(a, n-1, target);

}
```

The function could be used in the following way (printing `false`):

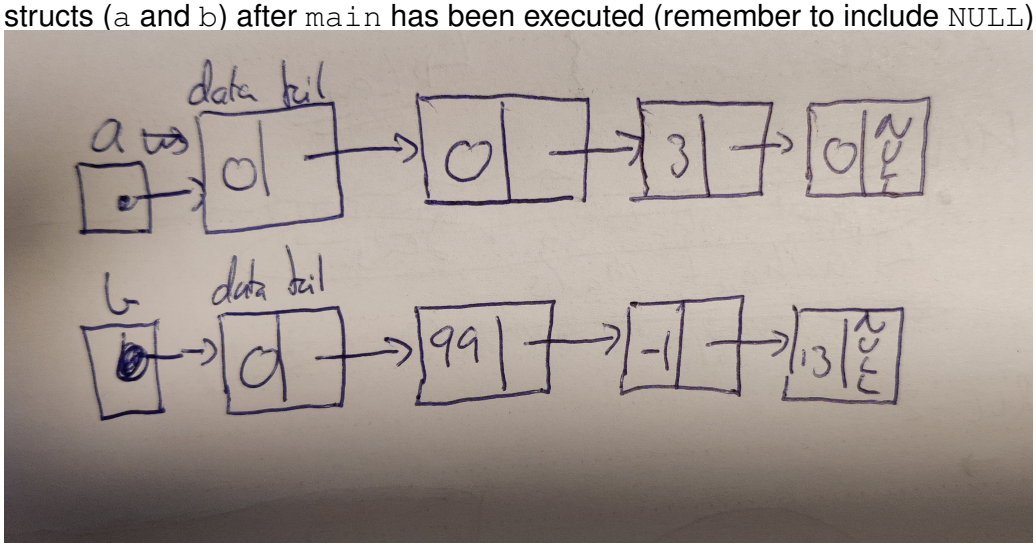
```
int main(void) {
  int a[] = {1,2,3,4,5};
  if (search(a, 5, 13))
    printf("true");
  else
    printf("false");
}
```

6 Bonus

Question 6.1 [10]

Consider the following program fragment:

```c
typedef struct node {
   int data;
   struct node *tail;
} node;


void modify(node* a, node* b) {
  if(a != NULL && b != NULL) {

    if(a->data < b->data)
      a->data = b->data;
    else if(b->data >0)
      b->data = a->data;

    modify(a->tail, b);
  }
}


node* new_node(int v, node* l) {
  node *n = (node*) malloc(sizeof(node));
  n->data = v;
  n->tail = l;
  return n;
}
```

```c
int main () {
  node* a = new_node(0,
              new_node(-1,
               new_node(3,
                new_node(0,NULL)
                )));

  node* b = new_node(0,
              new_node(99,
               new_node(-1,
                new_node(13,NULL)
                )));

  modify(a,b);
  // draw a and b here
}
```

Draw, in the same way as drawn in the lectures and question 4.1, the values of the allocated structs (`a` and `b`) after `main` has been executed (remember to include `NULL`):